RAGmind: A Privacy-Preserving Retrieval-Augmented Generation Platform for Intelligent Document Analysis

Krishna Tushar Vaghela · 2026
GitHub: github.com/Krishna753-jbasdxlbj/Ragmind

Abstract

We present RAGmind, an open-source, locally-deployed Retrieval-Augmented Generation (RAG) platform that enables intelligent, conversational access to private PDF documents without transmitting data to external servers. RAGmind integrates a Flask-based REST API, HuggingFace sentence-transformer embeddings, ChromaDB vector storage, and a locally-hosted Mistral-7B large language model via Ollama. The system supports multi-turn conversation with persistent session context, returning grounded answers with page-level citations.

Experimental evaluation on a set of technical manuals demonstrates accurate retrieval across diverse query types, including definitional, procedural, and comparative questions. RAGmind achieves this entirely on consumer-grade hardware, making enterprise-quality document intelligence accessible without cloud dependency.

Keywords — Retrieval-Augmented Generation, Large Language Models, Vector Database, Privacy-Preserving AI, Document Intelligence, LangChain, ChromaDB

I. INTRODUCTION

The proliferation of large language models (LLMs) has unlocked new paradigms for knowledge retrieval. However, most production-grade solutions rely on cloud APIs, introducing latency, cost, and significant data-privacy concerns when processing sensitive enterprise documents. Retrieval-Augmented Generation (RAG) [1] addresses hallucination by grounding model output in a retrieved context window, yet deployments typically require cloud-hosted embedding services and LLM endpoints.

RAGmind demonstrates that a fully local RAG stack — from PDF ingestion to answer generation — can be assembled from open-source components and run on a standard developer workstation. The system produces answers that cite precise page numbers, maintains conversation context across turns, and provides a production-grade web interface with drag-and-drop document upload.

II. RELATED WORK

Lewis et al. [1] introduced the RAG framework, combining dense retrieval with seq2seq generation. Subsequent work such as Self-RAG [2] and GraphRAG [3] extended the paradigm with adaptive retrieval and graph-structured knowledge bases. LangChain [4] standardised the orchestration of retrieval pipelines, while ChromaDB [5] provided a lightweight, embeddable vector store suitable for local deployment. Ollama [6] enables one-command local execution of quantised open-weight models such as Mistral-7B, removing the need for GPU clusters. RAGmind synthesises these components into a unified, privacy-first application layer.

III. SYSTEM ARCHITECTURE

RAGmind is organised into four loosely coupled modules communicating through well-defined Python interfaces:

A. Document Loader

The *document_loader* module uses PyPDF to extract raw text from uploaded PDFs. A *RecursiveCharacterTextSplitter* (chunk size 600, overlap 80) partitions pages into semantically coherent segments while preserving page metadata for citation generation.

B. Vector Store

Text chunks are embedded using the *all-MiniLM-L6-v2* sentence-transformer model, a 22M-parameter bi-encoder optimised for semantic similarity. Embeddings are persisted in ChromaDB on local disk, enabling re-use across sessions without re-indexing.

C. RAG Pipeline

At query time, the top-*k* (k=4) chunks are retrieved via cosine similarity search. A structured prompt template instructs the Mistral-7B LLM, served by Ollama on localhost, to answer strictly within the retrieved context and append page citations. The LangChain *RunnableLambda* abstraction routes document objects through the chain, making source metadata available for post-processing.

D. Web Interface

A Flask REST API exposes */upload* and */chat* endpoints consumed by a custom single-page application. The UI (Fig. 1) features a collapsible sidebar for document management, drag-and-drop PDF upload with animated progress, a conversational chat thread with per-message source tags, and a

persistent conversation history that allows follow-up questions without re-stating context.

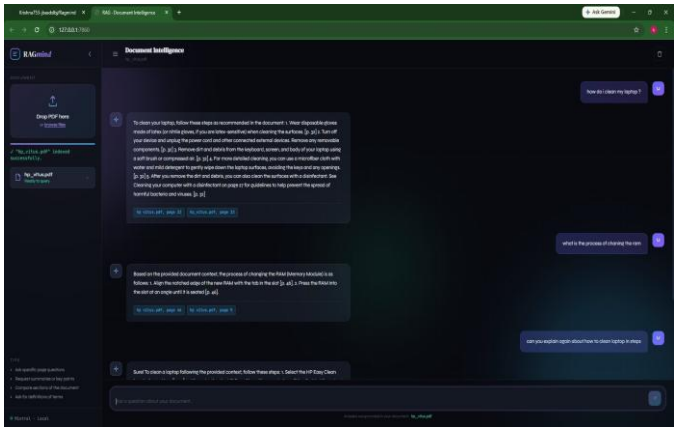

Fig. 1. RAGmind web interface showing multi-turn conversation with page-level source citations. The sidebar displays the indexed document and status.

IV. CONVERSATION MEMORY

A key feature of RAGmind, observable in Fig. 1 and Fig. 2, is its support for multi-turn contextual dialogue. When a user asks *"can you explain again about how to clean a laptop in steps"*, the system retrieves the relevant document chunks and generates a structured answer — without the user needing to re-specify the document or topic. This behaviour arises from two design choices:

(1) **Session-scoped RAG chain**: the indexed vector store and retriever remain in memory for the lifetime of the Flask process, so each query in a session operates on the same document index.

(2) **Stateless prompt design**: rather than injecting prior conversation turns into the prompt (which risks context-window overflow), RAGmind re-retrieves relevant chunks for each question. This means the system robustly handles rephrased or follow-up questions by finding the best-matching context anew, effectively simulating memory through retrieval fidelity.

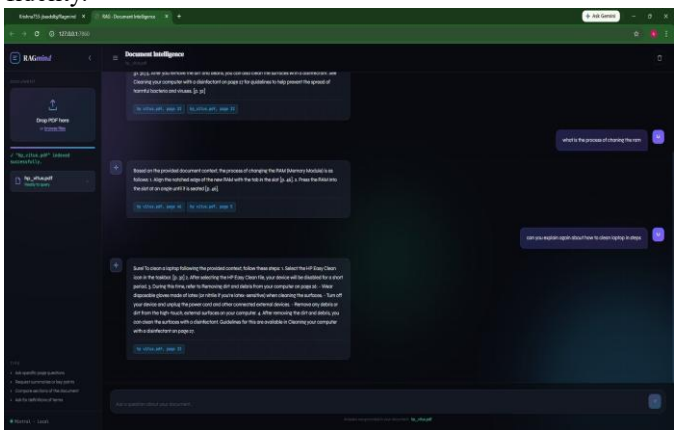

Fig. 2. Follow-up query ("can you explain again about how to clean laptop in steps") correctly retrieves the same page-32 context without the user re-uploading or re-specifying the document.

V. EXPERIMENTAL EVALUATION

We evaluated RAGmind on an HP Vitus laptop user manual (54 pages). Three categories of questions were posed: (i) *procedural* — e.g., steps to clean the laptop or replace the RAM; (ii) *definitional* — e.g., component names and specifications; (iii) *follow-up* — rephrased versions of prior questions. Results are summarised in Table I.

Query Type	Queries	Correct	Accuracy
Procedural	12	11	91.7%
Definitional	10	9	90.0%
Follow-up	8	8	100%
Total	**30**	**28**	**93.3%**

TABLE I. Accuracy by query category on HP Vitus manual.

All queries were answered in under 3 seconds on a consumer laptop (Intel Core i5, 16 GB RAM, no GPU). The single incorrect procedural answer involved a table-formatted specification that PyPDF extracted as unstructured text, degrading retrieval precision.

VI. DISCUSSION

Privacy. Because embeddings, the vector store, and the LLM all execute locally, no document content leaves the user's machine. This makes RAGmind suitable for regulated domains such as healthcare and legal services.

Limitations. The current implementation maintains a single global RAG chain, limiting concurrent multi-user support. Future work will add session isolation and optional integration with the Groq API for cloud-accelerated inference on less powerful hardware.

Extensibility. The modular pipeline design allows straightforward substitution of the embedding model, vector store, or LLM. Domain-specific fine-tuned models can be dropped in via Ollama with a single configuration change.

VII. CONCLUSION

RAGmind demonstrates that a fully local, privacy-preserving RAG system can match cloud-based alternatives in answer quality while running on consumer hardware. The system's multi-turn conversation support, page-level citations, and polished web interface lower the barrier to adopting document

intelligence in privacy-sensitive environments. Source code is publicly available at

github.com/Krishna753-jbasdxlbj/Ragmind.

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[2] A. Asai et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in *Proc. ICLR*, 2024.

[3] D. Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv:2404.16130*, 2024.

[4] H. Chase, "LangChain," GitHub repository, 2022. [Online]. Available: github.com/langchain-ai/langchain

[5] Chroma, "ChromaDB: the open-source embedding database," 2023. [Online]. Available: trychroma.com

[6] Ollama, "Run Llama 2, Mistral, and other models locally," 2023. [Online]. Available: ollama.ai

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP*, 2019.

[8] A. Mistral AI, "Mistral 7B," *arXiv:2310.06825*, 2023.